

Reflection Report

IT Student Management System

Avishek Pandey

IT Project Management

Prof. Deevya Seengal

Introduction

This report is my personal reflection on the IT Student Management System project that I completed over 13 weeks. In this report I will talk about what was hard, what I learned, what went well, and what I would do better next time. I have tried to be honest throughout.

1. What were the biggest challenges and how did I deal with them?

Doing everything by myself

The hardest part of this project was that I had to do everything alone. I had to plan the project, build the system, test it, and write all the documentation. In a real job, a team would do all of this together. Doing it alone was sometimes stressful.

The way I handled this was by using the sprint system from Agile. Instead of trying to think about all 13 weeks at once, I only focused on the next two weeks. At the end of each sprint, I checked what I had done and planned the next one. This helped me stay calm and on track.

Making sure each user only sees the right things

I needed to make sure that students could not access teacher pages, and teachers could not access admin pages. This sounds simple but it was actually tricky to set up properly in the code. My first attempt had the rules written separately on each page, which was messy and easy to make mistakes with.

I fixed this by putting all the access rules in one place in the code. After that, it was much cleaner and I felt more confident that no one could access something they were not supposed to.

Connecting all four parts of the system

Each module worked fine on its own, but making them all work together was harder than I expected. For example, when a teacher uploads a file, it needs to appear only for students in that specific class. Getting the data to flow correctly between the modules took a lot of debugging.

This taught me that database design needs to be done carefully at the very beginning. Some of my early design decisions had to be changed later, and that took extra time. Next time, I will think more carefully about how data connects before I start coding.

2. How did Agile help me and what did I learn from using it?

I had learned about Agile in class before, but actually using it on a real project was a completely different experience. Here is what I found most helpful:

- The sprint system kept me organised. Knowing I had two weeks to complete a specific module helped me focus instead of getting distracted.
- The review at the end of each sprint helped me catch problems early. For example, after Sprint 1 I realised my student profile design had a problem, and I was able to fix it before it caused bigger issues later.
- Agile lets you adjust the plan when things change. There were moments when something took longer than expected. Instead of panicking, I just updated the plan and moved on.

One thing I noticed is that Agile works much better with a team. When you are alone, there is no one to challenge your decisions or point out things you have missed. I tried to get around this by asking peers for feedback at the end of each sprint, but it is not quite the same as having a real team.

3. How did the project show what I have learned?

Before this project, I understood project management mostly from reading. This project made me actually do it, which is very different. Here are some specific things that showed what I have learned:

Planning properly

I made a clear project plan at the start that included the timeline, tasks, risks, and resources. I actually used it throughout the project and updated it when needed. All the features I planned were delivered on time, which shows the planning was realistic.

Using a project methodology

I ran four full sprints using Agile Scrum. Each sprint had a plan, a review, and a short reflection. I maintained a backlog and updated it after each sprint. This was not just theory — I actually did all of it in practice.

Technical skills

I built a working web application using Django and Python. I designed a database, set up user login, and made sure each user type could only access what they were allowed to. The system passed all four types of testing, which confirms it was built properly.

Handling risks

I made a risk table at the start that listed possible problems and how I would handle them. For example, I knew that trying to build too many features would be a risk for a solo

project, so I always built the most important things first. This meant that even when time got tight near the end, all the core features were already done.

4. Github Link

<https://github.com/1036025-hub/projectmanagement>

5. How did I handle the presentation and peer feedback?

The presentation

During the project pitch, I explained what the system was, who it was for, and how I planned to build it. I think it went okay. I was able to answer questions from peers and the supervisor without getting stuck, which showed I understood the project well.

One thing I could have done better was showing a visual mockup of the system instead of just talking about it. A picture would have helped people understand the design more quickly than my description did.

Peer feedback

After the pitch, peers said they were worried the interface might be too complicated, especially for students or teachers who are not very technical. I took this seriously and made some changes:

- Each user type only sees what is relevant to them. The student dashboard looks completely different from the teacher or admin dashboard.
- The navigation menu was made simpler and the labels were made clearer.
- Error messages were rewritten so they say something a normal person can understand, not a technical error code.

When peers tested the system later during user acceptance testing, they said it felt easy to use. That showed the changes I made based on their feedback actually worked.

6. What would I improve if I had more time?

Make it work on phones

Right now, the system is designed for desktop computers. But in countries like Nepal and India where this system is meant to be used, most people use their phones to access the internet. If I had more time, making the website look good on a phone would be my first priority.

Use a better database

I used SQLite because it is simple and quick to set up, which was good for a 13-week project. But it is not great when many people are using the system at the same time. I would switch to PostgreSQL, which can handle more users at once.

Put it online

At the moment the system only runs on my local computer. To make it actually useful for a school, it would need to be hosted online so anyone can access it from any device.

Add notifications

Currently students have to log in and check manually if new materials have been uploaded. It would be much more useful if the system sent an email or an in-app alert automatically when something new is added.

7. Overall Thoughts

Overall, I am happy with how this project went. I built a complete working system from scratch in 13 weeks by myself. That is something I am genuinely proud of.

Before this project, I thought the hardest part of building software was the coding. I was wrong. The hardest part was the planning, figuring out what to build, in what order, and making sure all the pieces would work together in the end. I had to learn this by actually doing it, not by reading about it.

I also learned that getting feedback from others is really important. Every time I showed my work to a peer, they noticed something I had missed. I was too close to the project to see it clearly. In future projects I will ask for feedback more often and earlier.

If I could do this project again, I would spend more time on the database design at the start, ask for feedback more regularly, and start the documentation earlier instead of leaving a lot of it to the final weeks. But even with those things, I feel like I learned a huge amount from this experience and I feel much more confident about managing a project in the future.

References

- Beck, K., et al. (2001). Manifesto for Agile Software Development. <https://agilemanifesto.org>
- Django Software Foundation. (2024). Django Documentation. <https://docs.djangoproject.com>
- Gibbs, G. (1988). Learning by Doing: A Guide to Teaching and Learning Methods. Oxford Polytechnic.
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. <https://scrumguides.org>
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.